

概述

DBMS的特色：数据独立性、数据库语言、事务处理

数据模型分类：概念模型、逻辑模型、物理模型

逻辑数据独立性：外模式映射到模式映射

物理数据独立性：模式到内模式映射

事务属性：原子性、一致性、隔离性、持久性

对数据库物理存储方式的描述称为内模式或存储模式

外模式是数据库用户直接接触和操作的数据逻辑表示，它定义了用户可以看到和操作的数据部分，以及相应的数据结构、数据操作和数据约束

SQL

```
#select字符串拼接
concat()
#select去重
distinct
#where查看包含字符
like
#where语句中多个值
in
#where 两个数值之间
between
#order by排序
desc #降序
asc #升序
#select date格式化
date_format(hire_date, '%m-%d/%Y')
#计算天数
datediff()
```

DDL

```
#创建数据库
create database database_name;
create database database_name character set utf8mb4;

#查看创建数据库代码
show create database database_name;

#创建数据库后修改数据库
alter database database_name character set utf8mb4;

#删除数据库
drop database if exists database_name;
```

```
#查看表的字段和数据类型
describe table_name;
show columns from table_name;

#查看表的字符集和存储引擎
show table status where name = 'table_name';

#添加表的字段
alter table table_name add field_name type;

#修改字段的属性
alter table table_name modify field_name new_type;

#修改字段的名称
alter table table_name change field_name new_name type;

#表重命名
rename table table_name to new_name;

#删除表
drop table if exists table_name;

#删除表的内容
delete from table_name; #支持回滚
truncate table table_name; #不支持回滚

#事务
start transaction; #开启事务
rollback; #回滚
commit; #提交事务
```

DML

```
#添加数据
insert into table_name (field_list) values (values_a),(values_b);
insert into table_name (field_list) values (values_list);

#更新数据
update table_name set field a = value a, field b = value b where condition;

#删除数据
delete from tb_name where condition;
```

DCL

```
#创建账号
create user 'anjingnan'@'%' identified by 'ddd';

#查看用户
select Host, User from mysql.user;

#查看指定用户
select Host, User from mysql.user where Host = '%';
```

```

select Host, User from mysql.user where name = 'anjingnan';

#修改用户名
rename user 'anijingnan' to 'ekaf';

#修改密码
alter user user( ) identified by 'aaa';
alter user 'anijingnan'@'%' identified by 'aaa';

\q

mysql -u anjingnan -pddd;
mysql -u mukana -paaa;

#删除账号
drop user 'anijingnan'@'%';

#授予权限
grant select on northwind.* to 'anijingnan'@'%';

#回收权限
revoke select on table_name from user_name;

#查看权限
show privileges;
show grants;

```

约束

```

#建表时
create table tb_null (
  id int not null,
  name varchar(15) not null,
  email varchar(25),
  salary decimal(10,2)
);

#建表后
alter table tb_null modify column email varchar(25) not null;

#删除约束
alter table tb_null modify column email varchar(25) null;

```

在 SQL 中，约束是用于确保数据库中数据的完整性和一致性的规则。约束可以应用于列（列级约束）或整个表（表级约束）。以下是一些常见的约束类型以及它们在列级和表级中的应用：

常见约束类型：

1. **主键 (PRIMARY KEY)**：确保一列中的所有值都是唯一的，并且不为空。
2. **外键 (FOREIGN KEY)**：确保一列中的值必须在另一个表的主键列中存在。
3. **唯一 (UNIQUE)**：确保一列中的所有值都是唯一的。
4. **非空 (NOT NULL)**：确保一列中的所有值都不为空。
5. **检查 (CHECK)**：确保一列中的值满足特定条件。

列级约束：

列级约束是直接定义在列的定义中的。当创建表时，可以在列的定义之后直接指定约束。例如：

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    salary DECIMAL(10, 2) CHECK (salary > 0)  
);
```

在上面的例子中，`id` 列有一个主键约束，`first_name` 和 `last_name` 列有非空约束，`email` 列有唯一约束，`salary` 列有一个检查约束。

表级约束：

表级约束是应用于整个表的约束，而不是单个列。它们通常在所有列的定义之后指定。例如：

```
CREATE TABLE employees (  
    id INT,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    email VARCHAR(100),  
    salary DECIMAL(10, 2),  
    PRIMARY KEY (id),  
    FOREIGN KEY (department_id) REFERENCES departments(id),  
    UNIQUE (email),  
    CHECK (salary > 0)  
);
```

在上面的例子中，`PRIMARY KEY` 和 `UNIQUE` 约束应用于 `id` 和 `email` 列，但它们是作为表级约束在所有列定义之后指定的。`FOREIGN KEY` 约束也是表级约束，它引用了另一个表的主键。

注意事项：

- 列级约束和表级约束在功能上是等价的，只是语法上的不同。
- 在修改表结构时，使用表级约束可能更方便，因为可以一次性修改多个列的约束。
- 某些数据库管理系统可能不支持某些类型的列级约束，例如 MySQL 不支持列级的外键约束。
- 约束有助于维护数据的完整性，但在设计数据库时应该谨慎使用，以避免不必要的性能开销。

在 SQL 中，您可以在创建表时添加约束，也可以在表创建之后使用 `ALTER TABLE` 语句来添加或删除约束。以下是一些常用的添加和删除约束的例子：

添加约束

创建表时添加约束：

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    salary DECIMAL(10, 2) CHECK (salary > 0),  
    department_id INT,  
    FOREIGN KEY (department_id) REFERENCES departments(id)  
);
```

使用 ALTER TABLE 添加约束：

```
-- 添加主键约束  
ALTER TABLE employees ADD PRIMARY KEY (id);  
-- 添加外键约束  
ALTER TABLE employees ADD FOREIGN KEY (department_id) REFERENCES  
departments(id);  
-- 添加唯一约束  
ALTER TABLE employees ADD UNIQUE (email);  
-- 添加非空约束  
ALTER TABLE employees MODIFY first_name VARCHAR(50) NOT NULL;  
-- 添加检查约束  
ALTER TABLE employees ADD CHECK (salary > 0);
```

请注意，添加主键约束时，如果表中已经存在数据，那么必须确保该列中的所有值都是唯一的且不为空。

删除约束

使用 ALTER TABLE 删除约束：

```
-- 删除主键约束  
ALTER TABLE employees DROP PRIMARY KEY;  
-- 删除外键约束  
ALTER TABLE employees DROP FOREIGN KEY fk_name; -- 其中 fk_name 是外键约束的名称  
-- 删除唯一约束  
ALTER TABLE employees DROP INDEX email; -- 在 MySQL 中，唯一约束也作为索引  
-- 删除非空约束  
ALTER TABLE employees MODIFY first_name VARCHAR(50);  
-- 删除检查约束  
ALTER TABLE employees DROP CHECK ck_name; -- 其中 ck_name 是检查约束的名称
```

在删除外键约束时，您需要知道数据库为该约束生成的名称。您可以通过查询数据库的系统表或信息模式来获取这个名称。在删除唯一约束时，由于在 MySQL 中唯一约束也作为索引，所以使用 `DROP INDEX` 语句来删除。

注意事项：

- 在删除或添加约束之前，请确保您了解这些更改对现有数据和应用程序的影响。
- 在生产环境中执行这些操作之前，最好先在开发或测试环境中测试更改。
- 确保您具有足够的权限来修改表结构。
- 不是所有的数据库管理系统都支持所有类型的约束。例如，MySQL 的 MyISAM 存储引擎不支持外键约束，而 InnoDB 存储引擎支持。请参考您使用的数据库管理系统的文档以了解支持的情况。

视图

#创建视图

```
create view v_name as
select field_list from tb_name;
```

可更新视图：并非所有视图都是可更新的。如果视图包含聚合函数、分组、连接、子查询、DISTINCT关键字或计算列，那么这个视图可能是只读的，不能被更新。

存储过程和存储函数

#创建存储过程

```
delimiter $
create procedure p_name(inout re_name)
begin
    select field into re_name from tb;
end $
delimiter ;
```

#调用存储过程

```
set @re_name = 'a';
call p_name(@re_name);
select @re_name
```

#创建存储函数

```
delimiter
create function f_name(r_name, type)
returns type
deterministic
contains sql
reads sql data
begin
    declare field_name type;

    if condition then
        return field_name;
    else
        return field_name;
    end if;
end $
delimiter ;
select f_name(r_name);
```

#查看存储过程代码

```

show create procedure p_name;

#查看存储函数代码
show create function f_name;

#查看存储过程状态
show procedure status like '';

#查看存储过程对象信息
select * from information_schema.routines where specific_name = 'p_name';

```

触发器

```

#创建数据库 tg（下面的操作都在这个数据库完成）
create database tg;

#检查 tg 数据库是否创建成功
show databases;

#创建表 tb_tg，包括两个字段：id 字段（int、主键、自增）和 note 字段（varchar(20)）
create table tb_tg(
    id int primary key auto_increment,
    note varchar(20)
);

#创建表 tb_tg_log，包括两个字段：id 字段（int、主键、自增）和 log 字段（varchar(40)）
create table tb_tg_log(
    id int primary key auto_increment,
    log varchar(40)
);

#创建表 employees，包括四个字段：employee_id, first_name, salary 和 manager_id，数据来自 northwind.employees
create table employees as
select employee_id, first_name, salary, manager_id
from northwind.employees;

#检查三个表的结构
describe tb_tg;
describe tb_tg_log;
describe employees;

#检查 employees 表的数据，确认有 northwind.employees 表对应的 107 条记录
select * from employees;

#创建名为 after_insert 的触发器，在 tb_tg 表插入数据后，向 tb_tg_log 表中插入日志信息
#日志信息的格式是：[datetime insert]: note, datetime 代表 insert 发生的当前日期和时间，下同
delimiter $
create trigger after_insert
after insert on tb_tg for each row
begin
    insert into tb_tg_log(log)
    values (concat('[', now(), ']insert: ', new.note));

```

```

end $
delimiter ;

#查看该触发器，确认创建成功
show triggers;

#在 tb_tg 表添加一条日志，内容随意
insert into tb_tg(note) value ('hello');

#查看 tb_tg 和 tb_tg_log 表的数据，确认触发器正常工作
select * from tb_tg;
select * from tb_tg_log;

#创建名为 after_delete 的触发器，在 tb_tg 表删除数据后，向 tb_tg_log 表中插入日志信息
#日志信息的格式是: [datetime delete]: note
delimiter $
create trigger after_delete
after delete on tb_tg for each row
begin
    insert into tb_tg_log(log)
    values (concat('[', now(), ']delete: ', old.note));
end $
delimiter ;

#查看该触发器，确认创建成功
show triggers;

#在 tb_tg 表删除一条日志
delete from tb_tg
where id = 3;
#查看 tb_tg 和 tb_tg_log 表的数据，确认触发器正常工作
select * from tb_tg;
select * from tb_tg_log;

#定义触发器 salary_check，在员工表添加新员工前检查其薪资是否大于他领导的薪资
#如果员工薪资大于其领导薪资，则报 sqlstate 为 45000 的错误
delimiter $
create trigger salary_check
after insert on employees for each row
begin
    declare mgr_sal double(10, 2);
    select salary into mgr_sal
    from employees
    where employee_id = new.manager_id;

    if new.salary > mgr_sal then
        signal sqlstate '45000';
    end if;
end $
delimiter ;

#查看该触发器，确认创建成功
show triggers;

#在员工表添加一条记录，做通过性测试，检查员工表的数据
insert into employees values
(207, 'ALLL', 10000, 100);

```


#在员工表添加一条记录，做失效性测试，检查员工表的数据

```
insert into employees values
```

```
(208, 'ANNN', 100000, 100);
```

#确认这个触发器能正常工作

.

#分别用三种方式查看上面创建的某个触发器

```
show triggers;
```

```
select * from information_schema.triggers where trigger_name = 'trigger_name';
```

```
show create trigger trigger_name;
```

关系代数

- 查找计算机系的学生【选择】
- $\sigma_{Sdept = 'CS'}(Student)$
- 查找计算机系的男生【选择】
- $\sigma_{Sdept = 'CS' \wedge Ssex = 'M'}(Student)$
- 查找学生信息，列出学号和姓名【投影】
- $\pi_{Sno, Sname}(Student)$
- 查找所有系部【投影】
- $\pi_{Sdept}(Student)$
- 查找计算机系和数学系的学生【并】
- $\sigma_{Sdept = 'CS'}(Student) \cup \sigma_{Sdept = 'Math'}(Student)$
- 查找选了 1002 号课程，但没选 3006 号课程的学生编号【差】
- $\pi_{Sno}(\sigma_{Cno = '1002'}(SC) - \sigma_{Cno = '3006'}(SC))$
- 查找已选课的学生和成绩信息【积】
- $(SC) \bowtie (Student)$
- 查找与 Elsa 同系的学生学号和姓名【积】
- $\pi_{S.Sno, S.Sname}(\sigma_{E.Sname = 'Elsa'}(\rho_S Student \bowtie S.Sdept = E.Sdept \rho_E Student))$
- 查找 3006 号课程的最高分【积】
- $\pi_{Grade}(\gamma; MAX(Grade) \rightarrow Grade (\sigma_{Cno = '3006'} SC))$
- 查找计算机系学生的 Sno 和 Ssex，重命名为 ID 和 Gender【重命名】
- $\pi_{ID, Gender}(\rho_{Sno \rightarrow ID, Ssex \rightarrow Gender}(\sigma_{Sdept = 'CS'} Student))$
- 将 SC 的 Grade 属性名修改为 Score【重命名】
- $\rho_{Grade \rightarrow Score} SC$
- 查找既选了 1002 号课程，又选了 3006 号课程的学生编号【交】
- $\sigma_{Cno = '1002' \vee Cno = '3006'} SC$
- 查找计算机系学生的选课情况，列出学号、姓名、课号和分数【连接】
- $\sigma_{Student.Sdept = 'CS'}(Student \bowtie SC)$
- 查找没选课的学生编号和姓名【左外连接】
- $\pi_{S.Sno, S.Sname}(\sigma_{SC.Sno = null}(\rho_S Student \bowtie S.Sno = SC.Sno SC))$
- 查找选修了所有课程的学生编号【除】
- $\pi_{Sno, Cno} SC \div \pi_{Cno} Course$
- 统计每个系的男生人数和女生人数【分组】
- $\pi_{Sdept, Ssex, Num}(\gamma_{Sdept, Ssex; COUNT(*) \rightarrow Num} Student)$
- 统计每位已选课学生的选课数和平均分【分组】
- $\pi_{Sno, Num, Avg_Grade}(\gamma_{Sno; COUNT(*) \rightarrow Num, AVG(Grade) \rightarrow Avg_Grade} SC)$

- 查找数学专业的女生【选择之交换律】
- $\sigma_{Ssex='M'}(\sigma_{Sdept='Math'} Student)$
- $\sigma_{Sdept='Math'}(\sigma_{Ssex='M'} Student)$
- 查找计算机系 19 岁男生【选择之合并律】
- $\sigma_{Sdept='CS'}(\sigma_{Sage=19} Student)$
- $\sigma_{Sdept='CS' \wedge Ssex='F'} Student$
- 查找全体学生的学号和姓名【投影之归一律】
- $\pi_{Sno, Sname}(\pi_{Sno, Sname, Ssex} Student)$
- $\pi_{Sno, Sname} Student$
- 查找物理系、计算机系以及数学系的学生【并之结合律】
- $(\sigma_{Sdept='Physics'} Student \cup \sigma_{Sdept='CS'} Student) \cup \sigma_{Sdept='Math'} Student$
- $\sigma_{Sdept='Physics'} Student \cup (\sigma_{Sdept='CS'} Student \cup \sigma_{Sdept='Math'} Student)$
- 查找计算机系和物理系的所有女生【并之分配律】
- $\sigma_{Ssex='M' \wedge Sdept='CS'} Student \cup \sigma_{Ssex='M' \wedge Sdept='Physics'} Student$
- $\sigma_{Ssex='M'}(\sigma_{Sdept='CS' \vee Sdept='Physics'} Student)$
- 查找计算机系 19 岁男生【交之结合律】
- $(\sigma_{Sdept='CS'} Student \cap \sigma_{Sage=19} Student) \cap \sigma_{Ssex='M'} Student$
- $\sigma_{Sdept='CS'} Student \cap (\sigma_{Sage=19} Student \cap \sigma_{Ssex='M'} Student)$
- 查找计算机系 19 岁男生【交之分配律】
- $\sigma_{Sdept='CS'}(\sigma_{Ssex='M'} Student \cap \sigma_{Sage=19} Student)$
- $\sigma_{Sdept='CS'}(\sigma_{Sage=19} Student) \cap \sigma_{Sdept='CS'}(\sigma_{Ssex='M'} Student)$
- 查找学号为 PH-001，但没选 1002 课程，成绩为 88 分的学生【差之分配律】
- $\sigma_{Sno='PH-1001'} SC - \sigma_{Sno='PH-1001' \wedge Grade=88} SC$
- $\sigma_{Sno='PH-1001'}(SC - \sigma_{Cno='1002' \wedge Grade=88} SC)$
- 查询学生和成绩信息【连接之交换律】
- $Student \bowtie SC$
- $SC \bowtie Student$
- 查询学生、课程和成绩信息【连接之结合律】
- $Student \bowtie Course \bowtie SC$
- $Student \bowtie (Course \bowtie SC)$
- 查找学号为 PH-001 的学生和成绩信息【连接和选择之分配律】
- $\sigma_{Student.Sno='PH-001'}(Student \bowtie SC)$
- $\sigma_{Student.Sno='PH-001'} Student \bowtie \sigma_{SC.Sno='PH-001'} SC$
- 查询学号为 PH-001 且成绩为 88 分的学生和成绩信息【连接和选择之分配律】
- $\sigma_{Student.Sno='PH-001'} Student \bowtie \sigma_{SC.Sno='PH-001' \wedge Grade=88} SC$
- $\sigma_{Student.Sno='PH-001' \wedge SC.Grade=88}(Student \bowtie SC)$
- 查询学生和成绩信息，包括：学号、系部、课号和成绩【连接和投影之分配律】
- $\pi_{Sno, Sdept, Cno, Grade}(Student \bowtie SC)$
- $(\pi_{Sno, Sdept} Student) \bowtie (\pi_{Sno, Cno, Grade} SC)$